DATA STRUCTURES IMPORTANT QUESTIONS

UNIT – 1

1(a) Define Structure. Explain creation, accessing and initialization of structure with suitable examples. (4M)

Definition

A **structure** in C is a user-defined data type that allows grouping of variables of different data types under a single name.

It is used to store related information such as student details, employee data, etc.

Syntax

```
struct structure_name
{
   data_type member1;
   data_type member2;
};
```

Example (Structure Creation)

```
struct Student
{
   int roll;
   char name[20];
   float marks;
};
```

Initialization of Structure

Structure variables can be initialized when declared.

```
struct Student s1 = {101, "Ravi", 85.5};
```

Accessing Structure Members

Members of a structure are accessed using the **dot (.) operator**.

```
printf("%d", s1.roll);
printf("%s", s1.name);
printf("%f", s1.marks);
```

Conclusion

Thus, structures help in organizing different types of related data under one variable.

1(b) Define Data Structure. Explain the different types of Linear Data Structures along with their representations. (4M)

Definition

A **Data Structure** is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently.

Linear Data Structures

In linear data structures, elements are arranged **sequentially**, and each element is connected to the next element.

Types of Linear Data Structures

1. **Array**
    - Collection of elements of the same data type.
    - Stored in **contiguous memory locations**.
    - Accessed using index.
    - **Representation**

A[0] A[1] A[2] A[3]

2. **Linked List**
    - Collection of nodes connected using pointers.
    - Each node contains **data and a link to the next node.**
    - **Representation**

Data | Link → Data | Link → Data | NULL

3. **Stack**
    - Follows **LIFO (Last In First Out)** principle.
    - Operations: **Push and Pop.**
    - **Representation**

Top
|
[30]
[20]
[10]

4. **Queue**
    - Follows **FIFO (First In First Out)** principle.
    - Operations: **Enqueue and Dequeue.**
    - **Representation**

Front → [10] [20] [30] ← Rear

Conclusion

Linear data structures store data sequentially and include **arrays, linked lists, stacks, and queues**, each with specific representations and operations.

2(a) Illustrate the concept of Nested Structure with examples. (4M)

Definition

A **Nested Structure** is a structure that contains another structure as one of its members.
It is used when a structure needs to store complex data made up of smaller related structures.

Syntax

```
struct structure1
{
```

```
    data_type member;
    struct structure2 member_name;
};
```

Example

```
#include <stdio.h>

struct Date
{
    int day;
    int month;
    int year;
};

struct Student
{
    int roll;
    char name[20];
    struct Date dob;   // Nested structure
};

int main()
{
    struct Student s1 = {101, "Ravi", {10, 5, 2004}};

    printf("Roll: %d\n", s1.roll);
    printf("Name: %s\n", s1.name);
    printf("DOB: %d-%d-%d", s1.dob.day, s1.dob.month, s1.dob.year);

    return 0;
}
```

Explanation

Here, the **Date structure is nested inside the Student structure** to store the student's date of birth.

Conclusion

Nested structures help organize complex data by combining related structures inside another structure.

2(b) Write a C program using typedef to define a new data type for structure. (4M)

Definition

typedef is used in C to **create a new name (alias) for an existing data type**. It simplifies the declaration of structure variables.

Example Program

```
#include <stdio.h>

typedef struct
{
   int id;
   char name[20];
   float salary;
} Employee;

int main()
{
   Employee e1;

   e1.id = 1;
   printf("Enter Name: ");
   scanf("%s", e1.name);
   printf("Enter Salary: ");
   scanf("%f", &e1.salary);

   printf("\nEmployee Details\n");
   printf("ID: %d\n", e1.id);
   printf("Name: %s\n", e1.name);
```

```
    printf("Salary: %.2f", e1.salary);

    return 0;
}
```

Explanation

- typedef struct { ... } Employee; creates a **new data type called Employee**.
- Now we can declare variables simply as:

Employee e1;

instead of:

struct Employee e1;

Conclusion

Thus, typedef simplifies structure declarations and improves code readability.

3(a) When is an Array of Structures needed? Justify with an example. (4M)

Definition

An **array of structures** is used when we need to store **multiple records of the same structure type**.

It allows storing many structure variables under a single name using indexing, just like arrays.

When it is Needed

An array of structures is required when:

- We need to store **details of many objects of the same type**.
- Each object contains **multiple attributes**.
- Example: storing details of many **students, employees, books**, etc.

Syntax

struct structure_name array_name[size];

Example

```
#include <stdio.h>

struct Student
{
   int roll;
   char name[20];
   float marks;
};

int main()
{
   struct Student s[3];   // Array of structures

   for(int i=0;i<3;i++)
   {
      printf("Enter Roll, Name, Marks: ");
      scanf("%d %s %f",&s[i].roll,s[i].name,&s[i].marks);
   }

   printf("\nStudent Details\n");

   for(int i=0;i<3;i++)
   {
      printf("%d %s %.2f\n",s[i].roll,s[i].name,s[i].marks);
   }

   return 0;
}
```

Conclusion

Thus, an **array of structures is useful for storing multiple records of the same type efficiently.**

3(b) Write a C program to calculate total and average marks of a student using functions and structure. (4M)

Program

```
#include <stdio.h>

struct Student
{
   int m1, m2, m3;
};

void calculate(struct Student s)
{
   int total;
   float average;

   total = s.m1 + s.m2 + s.m3;
   average = total / 3.0;

   printf("Total Marks = %d\n", total);
   printf("Average Marks = %.2f\n", average);
}

int main()
{
   struct Student s1;

   printf("Enter marks of 3 subjects: ");
   scanf("%d %d %d", &s1.m1, &s1.m2, &s1.m3);

   calculate(s1);

   return 0;
}
```

Explanation

- A **structure Student** stores marks of three subjects.
- A **function calculate()** computes total and average.
- The structure variable is passed to the function as an argument.

Output Example

Enter marks of 3 subjects: 80 85 90
Total Marks = 255
Average Marks = 85.00

4(a) Distinguish between Array, Structure and Union with examples. (4M)

Feature	Array	Structure	Union
Definition	Collection of elements of the **same data type**	Collection of variables of **different data types**	Similar to structure but **shares same memory location**
Memory	Each element has **separate memory**	Each member has **separate memory**	All members **share the same memory**
Data Type	Same data type only	Different data types allowed	Different data types allowed
Usage	Used to store list of values	Used to store record of related data	Used to save memory

Example

Array

```
int a[5];
```

Structure

```
struct Student
{
```

```
    int roll;
    char name[20];
    float marks;
};
```

Union

```
union Data
{
    int i;
    float f;
    char c;
};
```

Conclusion

- **Array** → same data type elements
- **Structure** → different data types stored together
- **Union** → different data types sharing the same memory location

---

4(b) Write a C program to declare an enum for days of the week and print any given day. (4M)

Program

```
#include <stdio.h>

enum Week {Sunday, Monday, Tuesday, Wednesday, Thursday, Friday,
Saturday};

int main()
{
    enum Week day;

    day = Wednesday;
```

```
    printf("Selected Day Number: %d\n", day);
    printf("The Day is Wednesday\n");

    return 0;
}
```

Explanation

- enum Week defines constants for the **days of the week**.
- By default values start from **0**.
- Here Wednesday is assigned to variable day and printed.

Output

Selected Day Number: 3
The Day is Wednesday

5(a) Describe Self-Referential Structures with an example. (4M)

Definition

A **self-referential structure** is a structure that contains a **pointer to another variable of the same structure type**.

It is mainly used in **dynamic data structures** such as **linked lists, trees, and graphs**.

Syntax

```
struct structure_name
{
    data_type member;
    struct structure_name *pointer_name;
};
```

Example

```
#include <stdio.h>
```

```
struct Node
{
   int data;
   struct Node *next;
};

int main()
{
   struct Node n1, n2;

   n1.data = 10;
   n1.next = &n2;

   n2.data = 20;
   n2.next = NULL;

   printf("First Node Data: %d\n", n1.data);
   printf("Second Node Data: %d\n", n2.data);

   return 0;
}
```

Explanation

- The structure **Node contains a pointer to another Node**.
- This allows linking multiple nodes together.
- It forms the basic concept of a **linked list**.

Conclusion

Thus, **self-referential structures allow structures to reference themselves and create linked data structures**.

5(b) Write a C program to store and display details of 5 students using an array of structures. (4M)

Program

```
#include <stdio.h>

struct Student
{
   int roll;
   char name[20];
   float marks;
};

int main()
{
   struct Student s[5];
   int i;

   printf("Enter details of 5 students:\n");

   for(i=0;i<5;i++)
   {
      printf("Enter Roll, Name, Marks: ");
      scanf("%d %s %f", &s[i].roll, s[i].name, &s[i].marks);
   }

   printf("\nStudent Details:\n");

   for(i=0;i<5;i++)
   {
      printf("Roll: %d Name: %s Marks: %.2f\n",
      s[i].roll, s[i].name, s[i].marks);
   }

   return 0;
}
```

Explanation

- A **structure Student** stores roll number, name, and marks.
- An **array of structures s[5]** stores details of 5 students.
- A **loop is used to input and display student data**.

Conclusion

Thus, an **array of structures is useful for storing and managing multiple records efficiently.**

6(a) Write short notes on the following terms (4M)

i) Enum (Enumeration)

An **enum** is a user-defined data type in C used to assign **names to integral constants**, which improves program readability.

Syntax

```
enum enum_name {value1, value2, value3};
```

Example

```
#include <stdio.h>

enum Week {Sun, Mon, Tue, Wed, Thu, Fri, Sat};

int main()
{
   enum Week day;
   day = Wed;
   printf("%d", day);
   return 0;
}
```

Note: By default, enum values start from **0**.

---

ii) typedef

typedef is used to **create a new name (alias) for an existing data type**.
It makes the program easier to read and write.

Syntax

typedef existing_type new_name;

Example

```
typedef int Integer;


int main()
{
   Integer a = 10;
   printf("%d", a);
}
```

Here, **Integer acts as another name for int.**

---

iii) Bit Fields

Bit fields are used in structures to **allocate memory in terms of bits instead of bytes.**
They help **save memory** when storing small values.

Example

```
struct Data
{
   unsigned int a:2;
   unsigned int b:3;
};
```

Here:

- a occupies **2 bits**
- b occupies **3 bits**

Bit fields are commonly used in **hardware programming and memory optimization**.

6(b) Write a C program to create a structure named Student with roll number, name, and marks. Read and display the details. (4M)

Program

```
#include <stdio.h>

struct Student
{
   int roll;
   char name[20];
   float marks;
};

int main()
{
   struct Student s;

   printf("Enter Roll Number: ");
   scanf("%d", &s.roll);

   printf("Enter Name: ");
   scanf("%s", s.name);

   printf("Enter Marks: ");
   scanf("%f", &s.marks);

   printf("\nStudent Details\n");
   printf("Roll Number: %d\n", s.roll);
   printf("Name: %s\n", s.name);
   printf("Marks: %.2f\n", s.marks);

   return 0;
}
```

Explanation

- A **structure Student** is defined with roll number, name, and marks.
- User inputs student details.
- The program **displays the stored values** using the structure variable.

7(a) Explain the classification of Data Structures with suitable examples. (4M)

Definition

A **Data Structure** is a method of organizing and storing data in a computer so that it can be accessed and modified efficiently.

Classification of Data Structures

Data structures are mainly classified into **two types**:

1. Primitive Data Structures

Primitive data structures are the **basic data types** provided by a programming language.

Examples

- int
- float
- char
- double

Example:

```
int a = 10;
float b = 5.6;
char c = 'A';
```

---

2. Non-Primitive Data Structures

Non-primitive data structures are **derived from primitive data types** and can store multiple values.

They are further classified into:

a) Linear Data Structures

In linear data structures, elements are arranged **sequentially**.

Examples

- Array
- Linked List
- Stack
- Queue

Example representation:

10 → 20 → 30 → 40

---

b) Non-Linear Data Structures

In non-linear data structures, elements are **not arranged sequentially**.

Examples

- Trees
- Graphs

Example representation (Tree):

```
   A
  / \
 B   C
```

Conclusion

Thus, data structures are classified into **Primitive and Non-Primitive**, where non-primitive structures are further divided into **Linear and Non-Linear data structures**.

7(b) Write a C program to define a structure with bit fields. (4M)

Program

```
#include <stdio.h>

struct Data
{
   unsigned int a:2;
   unsigned int b:3;
   unsigned int c:3;
};

int main()
{
   struct Data d;

   d.a = 2;
   d.b = 5;
   d.c = 6;

   printf("Value of a = %u\n", d.a);
   printf("Value of b = %u\n", d.b);
   printf("Value of c = %u\n", d.c);

   return 0;
}
```

Explanation

- The structure **Data** contains three bit fields.
- a uses **2 bits**, b uses **3 bits**, and c uses **3 bits**.

- Bit fields help **reduce memory usage** by allocating only required bits.

Output Example

Value of a = 2
Value of b = 5
Value of c = 6

8(a) Demonstrate the concept of Structures and Functions. (4M)

Definition

A **structure** is a user-defined data type that groups different data types under one name.
A **function** is a block of code that performs a specific task.

Structures can be **passed to functions** to perform operations on structured data.

Example Program

```
#include <stdio.h>

struct Student
{
   int roll;
   float marks;
};

void display(struct Student s)
{
   printf("Roll Number: %d\n", s.roll);
   printf("Marks: %.2f\n", s.marks);
}

int main()
{
   struct Student s1;
```

```
printf("Enter Roll Number: ");
scanf("%d", &s1.roll);

printf("Enter Marks: ");
scanf("%f", &s1.marks);

display(s1);   // Passing structure to function

return 0;
}
```

Explanation

- A structure **Student** stores roll number and marks.
- The function **display()** receives the structure variable as a parameter.
- The function prints the student details.

Conclusion

Thus, **structures and functions together help organize and process complex data efficiently**.

8(b) Write a C program to store and display details of a book (title, author, price) using structure. (4M)

Program

```
#include <stdio.h>

struct Book
{
   char title[30];
   char author[30];
   float price;
};
```

```c
int main()
{
   struct Book b;

   printf("Enter Book Title: ");
   scanf("%s", b.title);

   printf("Enter Author Name: ");
   scanf("%s", b.author);

   printf("Enter Price: ");
   scanf("%f", &b.price);

   printf("\nBook Details\n");
   printf("Title: %s\n", b.title);
   printf("Author: %s\n", b.author);
   printf("Price: %.2f\n", b.price);

   return 0;
}
```

Explanation

- The structure **Book** contains title, author, and price.
- User inputs the book details.
- The program displays the stored information.

9(a) Define Union. Explain with suitable examples. (4M)

Definition

A **union** is a user-defined data type in C that allows storing **different data types in the same memory location.**

Unlike structures, **all members of a union share the same memory space,** so only **one member can hold a value at a time.**

Syntax

```
union union_name
{
   data_type member1;
   data_type member2;
};
```

Example Program

```
#include <stdio.h>

union Data
{
   int i;
   float f;
   char c;
};

int main()
{
   union Data d;

   d.i = 10;
   printf("Integer value: %d\n", d.i);

   d.f = 5.5;
   printf("Float value: %.2f\n", d.f);

   return 0;
}
```

Explanation

- The union **Data** contains three members: int, float, and char.
- All members share the **same memory location**.
- When a new value is assigned, it **overwrites the previous value**.

Conclusion

Thus, unions are used to **save memory when variables are not needed simultaneously.**

9(b) Write a C program to define a structure Employee that contains another structure Address (city and pincode). Read and display employee details. (4M)

Program

```
#include <stdio.h>

struct Address
{
   char city[20];
   int pincode;
};

struct Employee
{
   int id;
   char name[20];
   struct Address addr;
};

int main()
{
   struct Employee e;

   printf("Enter Employee ID: ");
   scanf("%d", &e.id);

   printf("Enter Name: ");
   scanf("%s", e.name);
```

```
printf("Enter City: ");
scanf("%s", e.addr.city);

printf("Enter Pincode: ");
scanf("%d", &e.addr.pincode);

printf("\nEmployee Details\n");
printf("ID: %d\n", e.id);
printf("Name: %s\n", e.name);
printf("City: %s\n", e.addr.city);
printf("Pincode: %d\n", e.addr.pincode);

return 0;
}
```

Explanation

- Two structures are defined: **Address** and **Employee**.
- The **Employee structure contains Address as a member**, forming a **nested structure**.
- The program reads and displays employee details.
